

Intelligent Freight Quote Generation System

Description · System Architecture · Detailed Process | React + Django + MongoDB + JWT + ML

1. Project Description

What is this project?

A web platform where a customer enters shipment details — from-place, to-place, weight, volume, cargo type and transport mode — and the system **instantly generates a freight price quote** with a full cost breakdown. No phone calls, no waiting, no manual calculation by an agent. Think of it as **“flight-ticket-style instant pricing, but for cargo shipping.”**

The problem it solves

- **Slow quoting:** today, freight prices are calculated manually by human agents — customers wait hours or days.
- **Inconsistent pricing:** two agents can quote two different prices for the same shipment.
- **No scalability:** the number of quotes a company can give is limited by staff count.
- **Wasted data:** years of past quotes are never used to make pricing smarter.

Who uses it? (two roles)

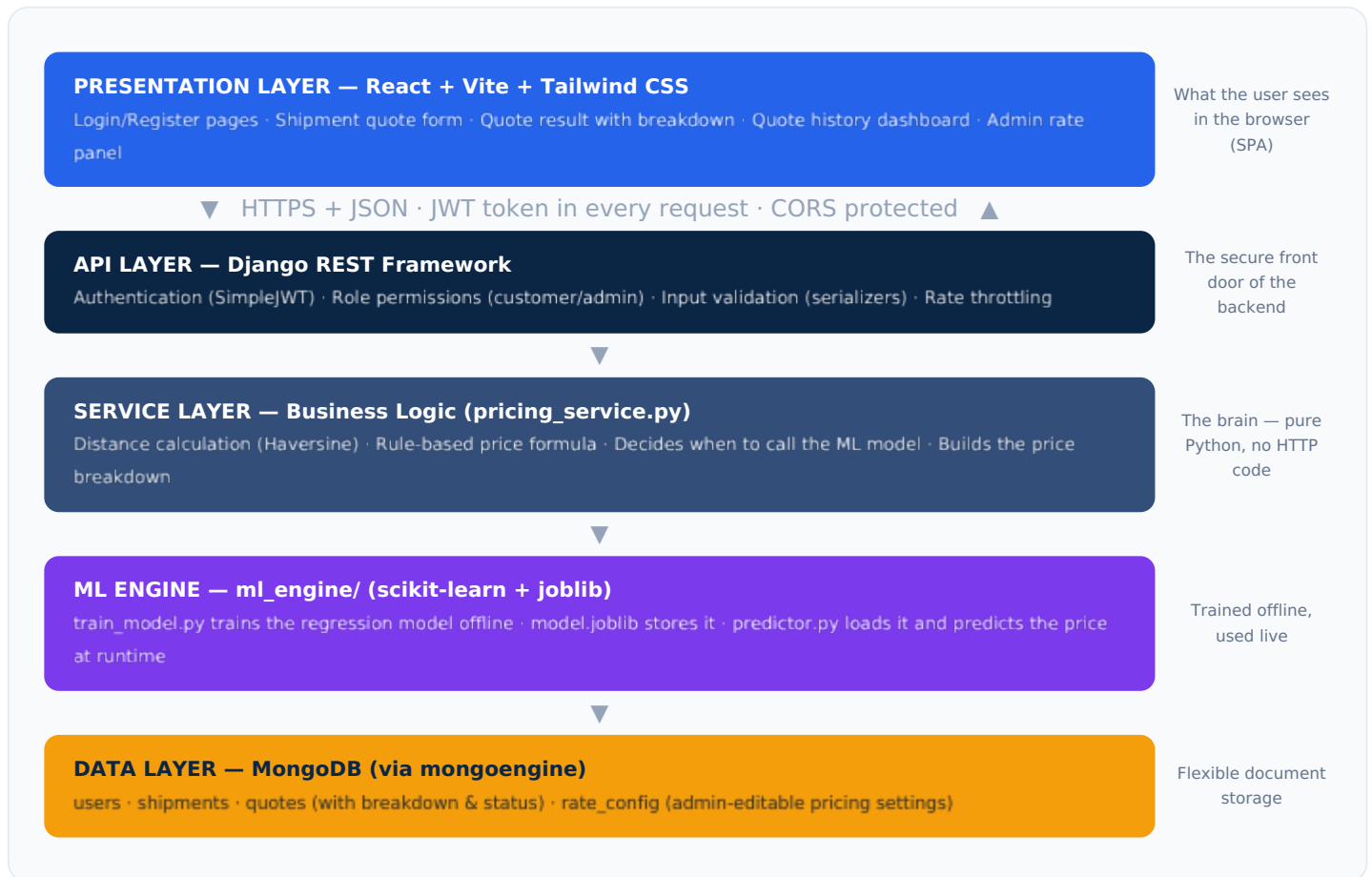
- **Customer:** registers, logs in, submits shipment details, receives instant quotes, views quote history.
- **Admin:** logs in to a protected panel and controls the pricing configuration — base rate per km, fuel surcharge %, cargo-type multipliers — without touching any code.

Where is the “intelligence”?

The price is generated by a **Pricing Engine** that evolves in two stages:

- **Stage 1 — Rule-based:** $\text{price} = \text{base rate} \times \text{distance} \times \text{cargo multiplier} \times \text{mode multiplier} + \text{fuel surcharge}$. Transparent and easy to verify.
- **Stage 2 — Machine Learning:** a regression model (scikit-learn) trained on freight data learns pricing patterns from distance, weight, cargo type, transport mode and season — the same idea behind OLA/Uber dynamic pricing. The system then compares rule price vs ML price and reports model accuracy (R^2 , RMSE).

2. System Architecture



Why this layered design? (the key teaching point)

- **Views never contain business logic.** The API layer only checks security and validates input, then calls the service layer.
- **The service layer is pure Python** — easy to test, easy to change, no web code inside.
- **The ML engine is isolated** — the model can be retrained and swapped without touching the website. If the model fails, the system falls back to rule-based pricing.
- **This is exactly how enterprise IT products are structured** — not a single big script.

Security built into the architecture

- Passwords stored only as hashes
- JWT access + refresh tokens with expiry
- CORS locked to the known frontend origin
- Role checks: customers can never open the admin rate panel
- Request throttling against abuse

Data flow in one line

React form → JWT-secured API → serializer validates → service calculates distance & price (rules or ML) → quote saved in MongoDB → JSON response → React shows the quote with breakdown.

3. Detailed Process — Step by Step

1

Customer registers and logs in

The customer creates an account. The backend hashes the password and issues a **JWT token**. Every following request carries this token, which proves who the user is and what role they have (customer or admin).

2

Customer fills the shipment form

Origin, destination, weight (kg), volume (m³), cargo type (standard / fragile / hazardous / perishable), and transport mode (road / rail / air / sea). The React form validates values before sending; the Django serializer validates them again on the server — double protection.

3

System calculates the distance

The backend converts origin and destination into coordinates and computes the distance using the **Haversine formula** (or a geocoding service). Distance is the single biggest factor in the freight price.

4

Pricing engine generates the price

Rule mode: base rate × distance × cargo multiplier × mode multiplier + fuel surcharge — using the values the admin configured.

ML mode: the trained regression model receives distance, weight, cargo type, mode and season, and predicts the price learned from data. The system records which model version produced each quote.

5

Customer receives the instant quote

The response shows the **total price and the full breakdown** — base cost, distance cost, fuel surcharge, cargo charge. The quote is stored in MongoDB with a status (draft / confirmed / expired) and an expiry date, exactly like real freight portals.

6

History and dashboard

Every customer sees their own past quotes (and only their own — enforced by role security). A dashboard shows charts such as quotes over time and average cost per route, pulled live from MongoDB.

7

Admin manages pricing — system stays current

The admin panel (admin-only, role-protected) lets the company update base rate, fuel %, and multipliers at any time — the very next quote uses the new values. Periodically the ML model is retrained on the growing quote history, so **the system keeps getting smarter with its own data**.